IRC Socket - Source Code

Group 4

23 October 2026

Contents

FILE	LINES	PAGE
bot/__init__.py	1	3
bot/__main__.py	4	3
bot/cli.py	31	3
bot/client.py	329	4
bot/commands.py	98	9
bot/data/facts.txt	5	11
bot/queries.py	204	11
bot/state.py	107	15
common/__init__.py	1	17
common/irc_message.py	133	17
common/numerics.py	27	19
common/protocol.py	44	20
common/stream.py	41	21
server/__init__.py	1	22
server/__main__.py	4	22
server/cli.py	30	22
server/client_session.py	182	23
server/errors.py	5	26
server/handlers.py	385	26
server/interfaces.py	44	32
server/server.py	218	33
server/state.py	195	37

bot/__init__.py

```python
"""IRC command bot."""
```

bot/__main__.py

```python
from .cli import main

if __name__ == "__main__":
    main()
```

bot/cli.py

```python
"""Command-line configuration for the bot."""

import argparse
import logging

from .client import BotClient


def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(description="IPv6 IRC bot")
    parser.add_argument("--host", default="fc00:1337::17")
    parser.add_argument("--port", type=int, default=6667)
    parser.add_argument("--name", default="SuperBot", dest="nickname")
    parser.add_argument("--channel", default="#hello")
    return parser


def main(argv: list[str] | None = None) -> None:
    args = build_parser().parse_args(argv)
    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s %(levelname)s %(name)s: %(message)s",
    )
    try:
        client = BotClient(args.host, args.port, args.nickname, args.channel)
    except ValueError as exc:
        build_parser().error(str(exc))
    try:
        client.run()
    except KeyboardInterrupt:
        client.stop()
```

bot/client.py

```python
"""IPv6 IRC bot client with registration, keep-alive and reconnect support."""

from __future__ import annotations

import logging
from collections import deque
from collections.abc import Callable
from functools import partial
import socket
import threading
import time

from common.irc_message import IRCMessage, message
from common.numerics import (
    RPL_ENDOFNAMES,
    RPL_NAMREPLY,
    RPL_WELCOME,
)
from common.protocol import irc_casefold, is_channel, valid_nickname
from common.stream import IRCStreamDecoder, IRCStreamError

from .commands import BotCommandProcessor, CommandAction
from .queries import BotQueries
from .state import BotState

logger = logging.getLogger(__name__)


class BotClient:
    REGISTRATION_TIMEOUT = 30.0
    IDLE_TIMEOUT = 30.0
    PONG_TIMEOUT = 30.0
    NAMES_TIMEOUT = 30.0
    MAX_PENDING_COMMANDS = 100
    TEXT_CHUNK_BYTES = 256

    def __init__(
        self,
        host: str = "fc00:1337::17",
        port: int = 6667,
        nickname: str = "SuperBot",
        channel: str = "#hello",
        reconnect_delay: float = 3.0,
    ) -> None:
        if not 1 <= port <= 65535:
            raise ValueError("port must be between 1 and 65535")
        self.host, self.port, self.nickname, self.channel = (
            host,
            port,
            nickname if valid_nickname(nickname) else "SuperBot",
            channel,
        )
        if not is_channel(self.channel):
            raise ValueError("channel must be a valid IRC channel name")
        self.reconnect_delay = reconnect_delay
        self.state = BotState()
        self.commands = BotCommandProcessor(self.nickname)
        self.queries = BotQueries()
        self._action_handlers: dict[CommandAction, Callable[[IRCMessage], None]] = {
            CommandAction.WAIT_FOR_NAMES: self._defer_command,
```

bot/client.py (continued)

```
            CommandAction.REQUEST_WHO: self._request_who,
            CommandAction.REQUEST_CHANNELS: partial(self._request_query, "LIST"),
            CommandAction.REQUEST_NETWORK: partial(self._request_query, "LUSERS"),
        }
        self.socket: socket.socket | None = None
        self.send_lock = threading.Lock()
        self.stopping = threading.Event()
        self._registered = False
        self._server_name = "s" * 63
        self._nick_attempt = 0
        self._names_started: float | None = None
        self._pending_commands: deque[IRCMessage] = deque()

    def run(self) -> None:
        while not self.stopping.is_set():
            try:
                self.run_once()
            except (ConnectionError, OSError, IRCStreamError) as exc:
                if not self.stopping.is_set():
                    logger.warning(
                        "Bot connection to [%s]:%s ended: %s", self.host, self.port, exc
                    )
            except Exception:
                logger.exception("Unexpected bot protocol error")
                raise
            finally:
                self.close()
            # A bounded delay avoids a tight reconnect loop when the server is
            # unavailable, while still allowing the bot to recover on its own.
            if not self.stopping.wait(self.reconnect_delay):
                continue

    def run_once(self) -> None:
        # Discard stale membership after reconnecting. JOIN events and the new
        # NAMES snapshot rebuild it; commands may arrive before NAMES completes.
        self.state.clear()
        self.queries.reset()
        self._names_started = None
        self._pending_commands.clear()
        self._registered = False
        self._nick_attempt = 0
        sock = socket.socket(socket.AF_INET6, socket.SOCK_STREAM)
        self.socket = sock
        if self.stopping.is_set():
            return
        sock.settimeout(1.0)
        sock.connect((self.host, self.port))
        started = last_received = time.monotonic()
        ping_sent: float | None = None
        ping_token: str | None = None
        decoder = IRCStreamDecoder()
        self.send(message("NICK", self.nickname))
        self.send(message("USER", self.nickname, "0", "*", self.nickname))
        while not self.stopping.is_set():
            self._check_timeouts(started, ping_sent)
            now = time.monotonic()
            if (
                self._registered
                and ping_sent is None
                and now - last_received >= self.IDLE_TIMEOUT
```

bot/client.py (continued)

```
            ):
                ping_token = f"bot-{now:.9f}"
                ping_sent = now
                self.send(message("PING", ping_token))
            try:
                data = sock.recv(4096)
            except socket.timeout:
                continue
            if not data:
                raise ConnectionError("server closed connection")
            self._check_timeouts(started, ping_sent)
            for incoming in decoder.feed(data):
                last_received = time.monotonic()
                if (
                    incoming.command == "PONG"
                    and incoming.params
                    and incoming.params[-1] == ping_token
                ):
                    ping_sent = None
                    ping_token = None
                self.handle(incoming)

    def _begin_names(self, channel: str) -> None:
        if not self.state.has_pending_names(channel):
            self.state.begin_names(channel)
            if irc_casefold(channel) == irc_casefold(self.channel):
                self._names_started = time.monotonic()

    def _check_timeouts(self, started: float, ping_sent: float | None) -> None:
        now = time.monotonic()
        if not self._registered and now - started >= self.REGISTRATION_TIMEOUT:
            raise TimeoutError("IRC registration timed out")
        if ping_sent is not None and now - ping_sent >= self.PONG_TIMEOUT:
            raise TimeoutError("server did not answer PING with matching PONG")
        if (
            self._names_started is not None
            and now - self._names_started >= self.NAMES_TIMEOUT
        ):
            self._names_started = None
            self._pending_commands.clear()
            raise TimeoutError("NAMES snapshot timed out")
        self.queries.check_timeouts(now)

    def send(self, msg: IRCMessage) -> None:
        with self.send_lock:
            sock = self.socket
            if sock is None:
                raise ConnectionError("bot is not connected")
            sock.sendall(msg.to_bounded_bytes())

    def handle(self, incoming: IRCMessage) -> None:
        command = incoming.command
        outgoing = self.queries.handle(incoming, self.nickname)
        if outgoing is not None:
            self._send_query_messages(outgoing)
            return
        if command in {"432", "433"} and not self._registered:
            self._nick_attempt += 1
            if self._nick_attempt > 10:
                raise ConnectionError("nickname registration rejected repeatedly")
```

bot/client.py (continued)

```
181             base = self.nickname if valid_nickname(self.nickname) else "SuperBot"
182             suffix = "_" + str(self._nick_attempt)
183             self.nickname = base[: 30 - len(suffix)] + suffix
184             self.commands.nickname = self.nickname
185             self.send(message("NICK", self.nickname))
186             return
187         if command == "PING":
188             token = incoming.params[-1] if incoming.params else ""
189             self.send(message("PONG", token))
190             return
191         if command == RPL_WELCOME:
192             self._registered = True
193             if incoming.prefix:
194                 self._server_name = incoming.prefix
195             if incoming.params:
196                 self.nickname = incoming.params[0]
197                 self.commands.nickname = self.nickname
198             # Registration completes only after the server sends 001; joining
199             # earlier can be rejected by a compliant server.
200             self._begin_names(self.channel)
201             self.send(message("JOIN", self.channel))
202             return
203         if command == "PRIVMSG":
204             self._handle_privmsg(incoming)
205             return
206         self._handle_channel_event(incoming)
207 
208     def _handle_channel_event(self, incoming: IRCMessage) -> None:
209         command = incoming.command
210         if command == RPL_NAMREPLY and len(incoming.params) >= 4:
211             # A large channel may be split across several 353 replies. Keep
212             # collecting names until 366 marks the end of this snapshot.
213             channel = incoming.params[2]
214             self._begin_names(channel)
215             names = incoming.params[3].lstrip(":").split()
216             self.state.add_names(
217                 channel,
218                 [name.lstrip("@+%~&") for name in names],
219             )
220             return
221         if command == RPL_ENDOFNAMES and len(incoming.params) >= 2:
222             channel = incoming.params[1]
223             self.state.finish_names(channel)
224             if irc_casefold(channel) == irc_casefold(self.channel):
225                 self._names_started = None
226                 pending = list(self._pending_commands)
227                 self._pending_commands.clear()
228                 for request in pending:
229                     self.handle(request)
230             return
231         if command == "JOIN" and incoming.prefix and incoming.params:
232             nickname = incoming.prefix.split("!", 1)[0]
233             channel = incoming.params[0]
234             if irc_casefold(nickname) == irc_casefold(self.nickname):
235                 self._begin_names(channel)
236             self.state.joined(channel, nickname)
237             return
238         if command == "PART" and incoming.prefix and incoming.params:
239             self.state.left(incoming.params[0], incoming.prefix.split("!", 1)[0])
240             return
```

bot/client.py (continued)

```
        if command == "QUIT" and incoming.prefix:
            self.state.removed(incoming.prefix.split("!", 1)[0])
            return
        if command == "NICK" and incoming.prefix and incoming.params:
            old = incoming.prefix.split("!", 1)[0]
            new = incoming.params[0]
            self.state.renamed(old, new)
            if irc_casefold(old) == irc_casefold(self.nickname):
                self.nickname = new
                self.commands.nickname = new
            return

    def _handle_privmsg(self, incoming: IRCMessage) -> None:
        if len(incoming.params) < 2 or not incoming.prefix:
            return
        sender = incoming.prefix.split("!", 1)[0]
        target, text = incoming.params[0], incoming.params[1]
        if irc_casefold(target) == irc_casefold(self.nickname):
            self._send_text(sender, self.commands.private_reply())
            return
        if irc_casefold(target) != irc_casefold(self.channel):
            return
        response = self.commands.channel_command(
            text,
            sender,
            self.state.members(target),
            members_complete=not self.state.has_pending_names(target),
        )
        if isinstance(response, CommandAction):
            self._action_handlers[response](incoming)
        elif response:
            self.send(message("PRIVMSG", target, response))

    def _defer_command(self, incoming: IRCMessage) -> None:
        if len(self._pending_commands) < self.MAX_PENDING_COMMANDS:
            self._pending_commands.append(incoming)
            return
        self.send(
            message(
                "PRIVMSG",
                incoming.params[0],
                "Channel roster is loading and the command queue is full; please retry.",
            )
        )

    def _request_who(self, incoming: IRCMessage) -> None:
        channel = incoming.params[0]
        self._send_query_messages(
            self.queries.start_who(
                channel, self.state.members(channel), self.nickname,
                self._server_name, time.monotonic(),
            )
        )

    def _request_query(self, command: str, incoming: IRCMessage) -> None:
        self._send_query_messages(
            self.queries.start(command, incoming.params[0], time.monotonic())
        )

    def _send_query_messages(self, outgoing: list[IRCMessage]) -> None:
```

bot/client.py (continued)

```
301         for reply in outgoing:
302             if reply.command == "PRIVMSG":
303                 self._send_text(reply.params[0], reply.params[1])
304             else:
305                 self.send(reply)
306 
307     def _send_text(self, target: str, text: str) -> None:
308         payload = text.encode("utf-8")
309         offset = 0
310         # Reserve space for nickname, username and IPv6 host fields added by
311         # the server. Split only at complete code points and retain all text.
312         while offset < len(payload):
313             text = payload[offset : offset + self.TEXT_CHUNK_BYTES].decode(
314                 "utf-8", errors="ignore"
315             )
316             self.send(message("PRIVMSG", target, text))
317             offset += len(text.encode("utf-8"))
318 
319     def close(self) -> None:
320         sock, self.socket = self.socket, None
321         if sock:
322             try:
323                 sock.close()
324             except OSError as exc:
325                 logger.debug("Bot socket close failed: %s", exc)
326 
327     def stop(self) -> None:
328         self.stopping.set()
329         self.close()
```

bot/commands.py

```
  1 """Pure Bot command decisions, kept independent from sockets."""
  2 
  3 from __future__ import annotations
  4 
  5 import random
  6 from datetime import datetime
  7 from enum import Enum, auto
  8 from pathlib import Path
  9 from common.protocol import irc_casefold
 10 
 11 
 12 class CommandAction(Enum):
 13     REQUEST_WHO = auto()
 14     REQUEST_CHANNELS = auto()
 15     REQUEST_NETWORK = auto()
 16     WAIT_FOR_NAMES = auto()
 17 
 18 
 19 class BotCommandProcessor:
 20     def __init__(
 21         self, nickname: str, facts_path: Path | None = None, rng: random.Random | None = None
 22     ) -> None:
 23         self.nickname = nickname
 24         self.rng = rng or random.Random()
 25         facts_path = facts_path or Path(__file__).with_name("data") / "facts.txt"
 26         try:
 27             self.facts = [
```

bot/commands.py (continued)

```
                line.strip()
                for line in facts_path.read_text(encoding="utf-8").splitlines()
                if line.strip() and "\x00" not in line
            ]
        except (OSError, UnicodeError):
            self.facts = []
        if not self.facts:
            self.facts = [
                "The network is made of packets, and packets travel one hop at a time."
            ]

    def channel_command(
        self,
        text: str,
        sender: str,
        channel_members: set[str],
        *,
        members_complete: bool = True,
    ) -> str | CommandAction | None:
        if not text.startswith("!"):
            return None
        command, _, argument = text[1:].partition(" ")
        command = command.casefold()
        if command in {"slap", "who"} and not members_complete:
            return CommandAction.WAIT_FOR_NAMES
        if command == "hello":
            return f"Hello, {sender}!"
        if command == "slap":
            return self._slap(argument.strip(), sender, channel_members)
        if command == "who":
            return CommandAction.REQUEST_WHO
        if command in {"channels", "network"}:
            if argument.strip():
                return f"Usage: !{command}"
            return (
                CommandAction.REQUEST_CHANNELS
                if command == "channels"
                else CommandAction.REQUEST_NETWORK
            )
        if command == "time":
            return "Bot time: " + datetime.now().astimezone().isoformat(
                sep=" ", timespec="seconds"
            )
        return None

    def _slap(self, target: str, sender: str, members: set[str]) -> str:
        folded_members = {irc_casefold(name): name for name in members}
        bot_key = irc_casefold(self.nickname)
        sender_key = irc_casefold(sender)
        eligible = {
            name
            for key, name in folded_members.items()
            if key not in {bot_key, sender_key}
        }
        # The no-argument form excludes both the bot and requester. An
        # explicit absent target falls back to the requester as required.
        target_key = irc_casefold(target)
        if target and target_key in {bot_key, sender_key}:
            return "Choose a channel member other than yourself or the Bot."
        if target and target_key in folded_members:
```

bot/commands.py (continued)

```
            chosen = folded_members[target_key]
        elif target:
            chosen = sender
        elif eligible:
            chosen = self.rng.choice(sorted(eligible))
        else:
            return "No eligible channel member to slap."
        return f"{sender} slaps {chosen} around a bit with a large trout!"

    def private_reply(self) -> str:
        return self.rng.choice(self.facts)
```

bot/data/facts.txt

```
Octopuses have three hearts.
The first email was sent in 1971.
IPv6 addresses are written in hexadecimal groups.
TCP delivers an ordered byte stream, not individual messages.
Honey can remain edible for a very long time when stored correctly.
```

bot/queries.py

```
"""IRC query state and response generation, independent of sockets and clocks."""

from __future__ import annotations

from dataclasses import dataclass, field

from common.irc_message import IRCMessage, message
from common.numerics import RPL_LIST, RPL_LISTEND, RPL_LUSERCLIENT
from common.protocol import irc_casefold, is_channel


@dataclass
class _WhoQuery:
    """One ISON batch group backing a channel's !who request."""

    channel: str
    started: float
    pending: int
    names: set[str] = field(default_factory=set)
    failed: bool = False


class BotQueries:
    WHO_TIMEOUT = 30.0
    QUERY_TIMEOUT = 30.0
    MAX_LISTED_CHANNELS = 10

    def __init__(self) -> None:
        self._who: _WhoQuery | None = None
        self._extra_queries: dict[str, tuple[str, float]] = {}
        self._listed_channels: list[tuple[str, str]] = []
        self._listed_count = 0

    def reset(self) -> None:
        """Discard all results and deadlines from a previous connection."""
        self._who = None
```

bot/queries.py (continued)

```
        self._reset_extra_queries()

    def _reset_extra_queries(self) -> None:
        self._extra_queries.clear()
        self._listed_channels.clear()
        self._listed_count = 0

    def start_who(
        self,
        channel: str,
        names: set[str],
        nickname: str,
        server_name: str,
        now: float,
    ) -> list[IRCMessage]:
        if self._who is not None:
            return []
        # miniircd's WHO reply has an unescaped IPv6 host parameter. ISON
        # confirms known members online without depending on that field.
        batches: list[list[str]] = []
        selected: list[str] = []
        for name in sorted(names):
            candidate = message("ISON", *(selected + [name]))
            reply = message(
                "303", nickname, " ".join(selected + [name]), prefix=server_name
            )
            # Budget the reply prefix as well as the request and parameter cap.
            if (
                len(selected) >= 14
                or len(candidate.to_bytes()) > 512
                or len(reply.to_bytes()) > 512
            ):
                batches.append(selected)
                selected = []
            selected.append(name)
        batches.append(selected or [nickname])
        self._who = _WhoQuery(channel=channel, started=now, pending=len(batches))
        return [message("ISON", *batch) for batch in batches]

    def start(self, command: str, channel: str, now: float) -> list[IRCMessage]:
        if command not in {"LIST", "LUSERS"}:
            raise ValueError("expected LIST or LUSERS")
        if command in self._extra_queries:
            return [
                message(
                    "PRIVMSG",
                    channel,
                    f"{command} query already in progress; please retry.",
                )
            ]
        self._extra_queries[command] = (channel, now)
        return [message(command)]

    def check_timeouts(self, now: float) -> None:
        if (
            self._who is not None
            and now - self._who.started >= self.WHO_TIMEOUT
        ):
            self._who = None
            # The caller reconnects so late replies cannot complete a new query.
```

bot/queries.py (continued)

```
            raise TimeoutError("ISON query timed out")
        for command, (_, started) in self._extra_queries.items():
            if now - started >= self.QUERY_TIMEOUT:
                self._reset_extra_queries()
                raise TimeoutError(f"{command} query timed out")

    def handle(self, incoming: IRCMessage, nickname: str) -> list[IRCMessage] | None:
        """Return outgoing messages, or None when this is not a query reply."""
        if (
            not incoming.params
            or irc_casefold(incoming.params[0]) != irc_casefold(nickname)
        ):
            return None
        outgoing = self._handle_extra_reply(incoming)
        if outgoing is not None:
            return outgoing
        who = self._who
        if who is None:
            return None
        if (
            incoming.command in {"263", "417", "421", "461"}
            and len(incoming.params) >= 2
            and incoming.params[1].upper() == "ISON"
        ):
            who.failed = True
            return self._finish_who_batch(who)
        if incoming.command == "303" and len(incoming.params) >= 2:
            who.names.update(incoming.params[-1].split())
            return self._finish_who_batch(who)
        return None

    def _finish_who_batch(self, who: _WhoQuery) -> list[IRCMessage]:
        who.pending -= 1
        if who.pending > 0:
            return []
        self._who = None
        if who.failed:
            return [
                message("PRIVMSG", who.channel, "WHO unavailable: server rejected ISON.")
            ]
        names = sorted(who.names)
        return [
            *(message("PRIVMSG", who.channel, f"WHO member: {name}") for name in names),
            message("PRIVMSG", who.channel, f"WHO complete: {len(names)} members"),
        ]

    def _handle_extra_reply(self, incoming: IRCMessage) -> list[IRCMessage] | None:
        params = incoming.params
        if incoming.command in {"263", "421", "461"} and len(params) >= 2:
            return self._reject_extra_query(params[1].upper())
        if incoming.command == RPL_LUSERCLIENT and len(params) >= 2:
            pending = self._extra_queries.pop("LUSERS", None)
            if pending is not None:
                return [message("PRIVMSG", pending[0], "NETWORK: " + params[-1])]
            return []
        return self._handle_list_reply(incoming)

    def _reject_extra_query(self, query: str) -> list[IRCMessage] | None:
        pending = self._extra_queries.pop(query, None)
        if pending is None:
```

bot/queries.py (continued)

```
157             return None
158         if query == "LIST":
159             self._listed_channels.clear()
160             self._listed_count = 0
161         return [
162             message(
163                 "PRIVMSG", pending[0], f"{query} unavailable: server rejected {query}."
164             )
165         ]
166 
167     def _handle_list_reply(self, incoming: IRCMessage) -> list[IRCMessage] | None:
168         params = incoming.params
169         pending = self._extra_queries.get("LIST")
170         if pending is None:
171             return None
172         if incoming.command == RPL_LIST and len(params) >= 4:
173             channel, count = params[1:3]
174             if (
175                 is_channel(channel)
176                 and len(count) <= 10
177                 and count.isascii()
178                 and count.isdigit()
179             ):
180                 self._listed_count += 1
181                 if len(self._listed_channels) < self.MAX_LISTED_CHANNELS:
182                     self._listed_channels.append((channel, count))
183             return []
184         if incoming.command == RPL_LISTEND and len(params) >= 2:
185             return self._finish_list(pending[0])
186         return None
187 
188     def _finish_list(self, destination: str) -> list[IRCMessage]:
189         del self._extra_queries["LIST"]
190         outgoing = [
191             message("PRIVMSG", destination, f"CHANNEL: {channel} ({count} users)")
192             for channel, count in self._listed_channels
193         ]
194         outgoing.append(
195             message(
196                 "PRIVMSG",
197                 destination,
198                 f"CHANNELS complete: {self._listed_count} channels "
199                 f"(showing {len(self._listed_channels)})",
200             )
201         )
202         self._listed_channels.clear()
203         self._listed_count = 0
204         return outgoing
```

bot/state.py

```python
"""Bot-side channel membership tracking."""

from __future__ import annotations

from common.protocol import irc_casefold


class BotState:
    def __init__(self) -> None:
        self.channels: dict[str, set[str]] = {}
        self._names_in_progress: dict[str, set[str]] = {}
        self._names_events: dict[str, list[tuple[str, tuple[str, ...]]]] = {}

    def joined(self, channel: str, nickname: str) -> None:
        self._record_event(channel, "joined", channel, nickname)
        # Keep IRC-equivalent keys stable while preserving nickname spelling.
        members = self.channels.setdefault(irc_casefold(channel), set())
        if self._matching_name(members, nickname) is None:
            members.add(nickname)

    @staticmethod
    def _matching_name(members: set[str], nickname: str) -> str | None:
        nickname_key = irc_casefold(nickname)
        return next((name for name in members if irc_casefold(name) == nickname_key), None)

    def left(self, channel: str, nickname: str) -> None:
        self._record_event(channel, "left", channel, nickname)
        members = self.channels.get(irc_casefold(channel))
        if members:
            existing = self._matching_name(members, nickname)
            if existing is not None:
                members.discard(existing)

    def renamed(self, old: str, new: str) -> None:
        for channel in self._names_in_progress:
            self._record_event(channel, "renamed", old, new)
        for members in self.channels.values():
            existing = self._matching_name(members, old)
            if existing is not None:
                members.discard(existing)
                members.add(new)

    def removed(self, nickname: str) -> None:
        for channel in self._names_in_progress:
            self._record_event(channel, "removed", nickname)
        for members in self.channels.values():
            existing = self._matching_name(members, nickname)
            if existing is not None:
                members.discard(existing)

    def members(self, channel: str) -> set[str]:
        return set(self.channels.get(irc_casefold(channel), set()))

    def clear(self) -> None:
        """Discard membership learned from a previous TCP connection."""
        self.channels.clear()
        self._names_in_progress.clear()
        self._names_events.clear()

    def _record_event(self, channel: str, event: str, *args: str) -> None:
```

bot/state.py (continued)

```
61          events = self._names_events.get(irc_casefold(channel))
62          if events is not None:
63              events.append((event, args))
64
65      def begin_names(self, channel: str) -> None:
66          self._names_in_progress[irc_casefold(channel)] = set()
67          self._names_events[irc_casefold(channel)] = []
68
69      def add_names(self, channel: str, names: list[str]) -> None:
70          if not self.has_pending_names(channel):
71              self.begin_names(channel)
72          pending = self._names_in_progress[irc_casefold(channel)]
73          for name in names:
74              if self._matching_name(pending, name) is None:
75                  pending.add(name)
76
77      def has_pending_names(self, channel: str) -> bool:
78          return irc_casefold(channel) in self._names_in_progress
79
80      def finish_names(self, channel: str) -> None:
81          channel_key = irc_casefold(channel)
82          if channel_key not in self._names_in_progress:
83              return
84          names = self._names_in_progress.pop(channel_key, set())
85          self.channels[channel_key] = names
86          # Apply events after the entire snapshot, including events preceding a
87          # later 353 chunk that still contains an old nickname.
88          events = self._names_events.pop(channel_key, [])
89          for event, args in events:
90              if event == "renamed":
91                  old, new = args
92                  existing = self._matching_name(names, old)
93                  if existing is not None:
94                      names.discard(existing)
95                      names.add(new)
96              elif event == "removed":
97                  existing = self._matching_name(names, args[0])
98                  if existing is not None:
99                      names.discard(existing)
100             else:
101                 getattr(self, event)(*args)
102
103     def replace_names(self, channel: str, names: list[str]) -> None:
104         # NAMES is authoritative for the snapshot; later events update this set.
105         self.begin_names(channel)
106         self.add_names(channel, names)
107         self.finish_names(channel)
```

common/__init__.py

```
"""Shared IRC protocol primitives."""
```

common/irc_message.py

```
"""RFC 2812 style IRC message parsing and serialization."""

from __future__ import annotations

from dataclasses import dataclass
from typing import Optional


class IRCParseError(ValueError):
    """Raised when an IRC line cannot be represented as an IRC message."""


@dataclass(frozen=True)
class IRCMessage:
    """A single IRC message without its CRLF terminator."""

    command: str
    params: tuple[str, ...] = ()
    prefix: Optional[str] = None

    def __post_init__(self) -> None:
        command = self.command.strip()
        if (
            not command
            or not command.isascii()
            or not (command.isalpha() or (len(command) == 3 and command.isdigit()))
        ):
            raise IRCParseError("command must be a non-empty token")
        object.__setattr__(self, "command", command.upper())
        object.__setattr__(self, "params", tuple(self.params))
        if self.prefix is not None and (
            not self.prefix
            or any(ch.isspace() or ch in "\r\n\x00" for ch in self.prefix)
        ):
            raise IRCParseError("prefix must be a non-empty token")
        if len(self.params) > 15:
            raise IRCParseError("IRC messages have at most 15 parameters")
        for param in self.params:
            if any(char in param for char in "\r\n\x00"):
                raise IRCParseError("parameters cannot contain CR, LF or NUL")

    @classmethod
    def parse(cls, line: str) -> "IRCMessage":
        """Parse one IRC line, keeping a colon-prefixed trailing field intact."""
        if line.endswith("\r\n"):
            line = line[:-2]
        if any(char in line for char in "\r\n\x00"):
            raise IRCParseError("IRC line cannot contain embedded CR, LF or NUL")
        if not line:
            raise IRCParseError("empty IRC line")

        prefix = None
        if line.startswith(":"):
            try:
                prefix, line = line[1:].split(" ", 1)
```

common/irc_message.py (continued)

```
            except ValueError as exc:
                raise IRCParseError("prefix is missing a command") from exc
            if not prefix:
                raise IRCParseError("empty prefix")

        line = line.lstrip(" ")
        if not line:
            raise IRCParseError("command is missing")
        if " :" in line:
            head, trailing = line.split(" :", 1)
            parts = head.split()
            parts.append(trailing)
        else:
            parts = line.split()
        if not parts:
            raise IRCParseError("command is missing")
        return cls(parts[0], tuple(parts[1:]), prefix)

    def serialize(self) -> str:
        """Return the wire representation without CRLF.

        Only the final parameter can contain spaces, so it is emitted as the
        IRC trailing field when needed. Keeping CRLF in ``to_bytes`` makes the
        message boundary explicit at the socket layer.
        """
        pieces = []
        if self.prefix is not None:
            pieces.append(f":{self.prefix}")
        pieces.append(self.command)
        if self.params:
            for index, param in enumerate(self.params):
                is_last = index == len(self.params) - 1
                if not is_last and (
                    not param
                    or any(ch.isspace() for ch in param)
                    or param.startswith(":")
                ):
                    raise IRCParseError(
                        "only the final parameter may contain spaces or start with ':'"
                    )
                if is_last and (
                    not param
                    or any(ch.isspace() for ch in param)
                    or param.startswith(":")
                ):
                    # The leading colon here is the wire delimiter. Preserve a
                    # second colon when the logical value itself starts with one.
                    pieces.append(":" + param)
                else:
                    pieces.append(param)
        return " ".join(pieces)

    def to_bytes(self) -> bytes:
        return (self.serialize() + "\r\n").encode("utf-8")

    def to_bounded_bytes(
        self, max_line_bytes: int = 512, *, truncate_trailing: bool = False
    ) -> bytes:
        """Bound outgoing lines, optionally shortening a human-readable reason."""
        payload = self.to_bytes()
```

common/irc_message.py (continued)

```
116         if len(payload) <= max_line_bytes:
117             return payload
118         if not truncate_trailing or not self.params:
119             raise IRCParseError("outgoing IRC line exceeds maximum length")
120         empty = IRCMessage(self.command, (*self.params[:-1], ""), self.prefix)
121         available = max_line_bytes - len(empty.to_bytes())
122         if available < 0:
123             raise IRCParseError("IRC routing fields exceed maximum length")
124         # Cutting bytes can split a UTF-8 code point; discard only that tail.
125         trailing = (
126             self.params[-1].encode("utf-8")[:available].decode("utf-8", errors="ignore")
127         )
128         bounded = IRCMessage(self.command, (*self.params[:-1], trailing), self.prefix)
129         return bounded.to_bytes()
130
131
132 def message(command: str, *params: str, prefix: Optional[str] = None) -> IRCMessage:
133     return IRCMessage(command, tuple(params), prefix)
```

common/numerics.py

```
  1 """IRC numeric reply constants used by this implementation."""
  2
  3 RPL_WELCOME = "001"
  4 RPL_YOURHOST = "002"
  5 RPL_CREATED = "003"
  6 RPL_MYINFO = "004"
  7 RPL_LUSERCLIENT = "251"
  8 RPL_LIST = "322"
  9 RPL_LISTEND = "323"
 10 RPL_NAMREPLY = "353"
 11 RPL_ENDOFNAMES = "366"
 12 RPL_TIME = "391"
 13 RPL_WHOREPLY = "352"
 14 RPL_ENDOFWHO = "315"
 15
 16 ERR_NOSUCHNICK = "401"
 17 ERR_NOSUCHCHANNEL = "403"
 18 ERR_CANNOTSENDTOCHAN = "404"
 19 ERR_NONICKNAMEGIVEN = "431"
 20 ERR_ERRONEUSNICKNAME = "432"
 21 ERR_NICKNAMEINUSE = "433"
 22 ERR_NOTONCHANNEL = "442"
 23 ERR_NOTREGISTERED = "451"
 24 ERR_ALREADYREGISTERED = "462"
 25 ERR_UNKNOWNCOMMAND = "421"
 26 ERR_NEEDMOREPARAMS = "461"
 27 ERR_INPUTTOOLONG = "417"
```

common/protocol.py

```
"""Protocol-level names and validation shared by client and server."""

IRC_COMMANDS = frozenset(
    {"NICK", "USER", "JOIN", "PART", "PRIVMSG", "NAMES", "QUIT", "PING", "PONG", "TIME", "ISON", "LIST", "LUSERS"}
)
CHANNEL_PREFIXES = ("#", "&")


def irc_casefold(name: str) -> str:
    """Apply RFC 1459 casemapping to nicknames and channel names."""
    return name.translate(str.maketrans("ABCDEFGHIJKLMNOPQRSTUVWXYZ[]\\^", "abcdefghijklmnopqrstuvwxyz{}|~"))


def is_channel(target: str) -> bool:
    """Return whether *target* is a valid channel name in this IRC subset.

    This subset accepts 2-50 characters, starting with '#' or '&', and only
    printable ASCII excluding spaces, commas, and colons. The ASCII restriction
    is a local choice; the 50-character limit follows RFC 2812. Sharing this
    check keeps the server and bot consistent about valid targets.
    """
    if not target or target[0] not in CHANNEL_PREFIXES:
        return False
    if not 2 <= len(target) <= 50:
        return False
    return all(33 <= ord(char) <= 126 and char not in ",:" for char in target)


def valid_nickname(nickname: str) -> bool:
    """Apply the compact nickname grammar supported by this server."""
    if not 1 <= len(nickname) <= 30:
        return False
    ascii_letters = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"
    special = "[]\\`_^{|}"
    if not (nickname[0] in ascii_letters or nickname[0] in special):
        return False
    return all(ch in ascii_letters + "0123456789" + special + "-" for ch in nickname)


def valid_username(username: str) -> bool:
    """Apply the local 1-30 ASCII character limit for user prefix fields."""
    return 1 <= len(username) <= 30 and all(
        33 <= ord(char) <= 126 and char not in "!@:" for char in username
    )
```

common/stream.py

```python
"""TCP stream framing for IRC's CRLF-delimited messages."""

from __future__ import annotations

from .irc_message import IRCMessage, IRCParseError


class IRCStreamError(ValueError):
    """Raised for malformed or over-sized IRC stream data."""


class IRCStreamDecoder:
    """Incrementally converts arbitrary TCP chunks into IRC messages."""

    def __init__(self, max_line_bytes: int = 512) -> None:
        self.max_line_bytes = max_line_bytes
        self._buffer = bytearray()

    def feed(self, data: bytes) -> list[IRCMessage]:
        if not isinstance(data, (bytes, bytearray, memoryview)):
            raise TypeError("data must be bytes-like")
        self._buffer.extend(data)
        messages: list[IRCMessage] = []
        while True:
            # TCP preserves byte order but not application write boundaries;
            # decode every complete CRLF line and retain any unfinished tail.
            delimiter = self._buffer.find(b"\r\n")
            if delimiter < 0:
                if len(self._buffer) > self.max_line_bytes:
                    self._buffer.clear()
                    raise IRCStreamError("IRC line exceeds maximum length")
                break
            raw = bytes(self._buffer[:delimiter])
            del self._buffer[: delimiter + 2]
            if len(raw) + 2 > self.max_line_bytes:
                raise IRCStreamError("IRC line exceeds maximum length")
            try:
                messages.append(IRCMessage.parse(raw.decode("utf-8")))
            except (UnicodeDecodeError, IRCParseError) as exc:
                raise IRCStreamError(str(exc)) from exc
        return messages
```

server/__init__.py

```
"""IPv6 IRC server implementation."""
```

server/__main__.py

```
from .cli import main

if __name__ == "__main__":
    main()
```

server/cli.py

```
"""Command-line configuration for the IRC server."""

import argparse
import logging

from .server import IRCServer


def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(description="IPv6 IRC server")
    parser.add_argument("--host", default="::", help="IPv6 listen address")
    parser.add_argument("--port", type=int, default=6667)
    parser.add_argument("--name", default="irc.local", dest="server_name")
    return parser


def main(argv: list[str] | None = None) -> None:
    args = build_parser().parse_args(argv)
    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s %(levelname)s %(name)s: %(message)s",
    )
    try:
        server = IRCServer(args.host, args.port, args.server_name)
    except ValueError as exc:
        build_parser().error(str(exc))
    try:
        server.serve_forever()
    except KeyboardInterrupt:
        server.stop()
```

server/client_session.py

```python
"""One TCP client session, isolated from protocol state and dispatch."""

from __future__ import annotations

import logging
from queue import Empty, Full, Queue
import socket
import threading
import time
from typing import TYPE_CHECKING

from common.irc_message import IRCMessage
from common.stream import IRCStreamDecoder, IRCStreamError

from .state import ClientState

if TYPE_CHECKING:
    from .server import IRCServer

logger = logging.getLogger(__name__)


class ClientSession:
    MAX_PENDING_MESSAGES = 256
    OUTPUT_QUEUE_TIMEOUT = 1.0

    def __init__(self, server: IRCServer, sock: socket.socket, address: tuple) -> None:
        self.server = server
        self.socket = sock
        self.address = address
        connected_at = server.monotonic()
        self.client = ClientState(
            self, address, last_activity=connected_at, connected_at=connected_at
        )
        self.decoder = IRCStreamDecoder()
        self.send_lock = threading.Condition()
        self.closed = threading.Event()
        self._outgoing: Queue[tuple[IRCMessage, bytes] | None] = Queue(
            self.MAX_PENDING_MESSAGES
        )
        self._writer: threading.Thread | None = None
        self._error_sent = False

    def start(self) -> None:
        with self.server.state.locked():
            # Shutdown snapshots clients under the same lock, so an accepted
            # socket cannot register after that snapshot and escape cleanup.
            if self.server.stopping.is_set():
                self.close()
                return
            self.server.state.add_client(self.client)
        try:
            threading.Thread(
                target=self.run, name=f"irc-client-{self.address}", daemon=True
            ).start()
        except BaseException:
            self.server.disconnect(
                self.client, "Session startup failed", announce=False
            )
            raise
```

server/client_session.py (continued)

```
61
62      def send(self, msg: IRCMessage) -> None:
63          self._enqueue(msg, wait_for_space=False)
64
65      def send_reply(self, msg: IRCMessage) -> None:
66          self.server.state.require_unlocked()
67          self._enqueue(msg, wait_for_space=True)
68
69      def _enqueue(self, msg: IRCMessage, *, wait_for_space: bool) -> None:
70          payload = msg.to_bounded_bytes(
71              truncate_trailing=msg.command in {"PART", "QUIT", "ERROR"}
72          )
73          deadline = time.monotonic() + self.OUTPUT_QUEUE_TIMEOUT
74          with self.send_lock:
75              while True:
76                  if self.closed.is_set():
77                      raise ConnectionError("session is closed")
78                  try:
79                      self._outgoing.put_nowait((msg, payload))
80                      return
81                  except Full as exc:
82                      if not wait_for_space:
83                          raise ConnectionError("client output queue is full") from exc
84                      remaining = deadline - time.monotonic()
85                      if remaining <= 0:
86                          raise ConnectionError("client output queue did not drain") from exc
87                      # Only a reply producer outside the state lock may wait.
88                      self.send_lock.wait(remaining)
89
90      def _write_loop(self) -> None:
91          try:
92              while not self.closed.is_set():
93                  item = self._outgoing.get()
94                  with self.send_lock:
95                      self.send_lock.notify_all()
96                  if item is None or self.closed.is_set():
97                      return
98                  msg, payload = item
99                  if msg.command == "PING":
100                     # Set before sendall so a fast PONG cannot beat this update.
101                     self.server.state.start_ping(self.client, self.server.monotonic())
102                 self.socket.sendall(payload)
103                 if msg.command == "ERROR":
104                     with self.send_lock:
105                         self._error_sent = True
106                         self.send_lock.notify_all()
107         except (ConnectionError, OSError) as exc:
108             logger.info("IRC write failed for %s: %s", self.address, exc)
109             self.server.disconnect(self.client, "Connection lost")
110         except Exception:
111             logger.exception("Unhandled IRC writer error from %s", self.address)
112             self.server.disconnect(self.client, "Output failure")
113
114     def run(self) -> None:
115         try:
116             self.socket.settimeout(1.0)
117             self._writer = threading.Thread(
118                 target=self._write_loop, name=f"irc-writer-{self.address}", daemon=True
119             )
120             self._writer.start()
```

server/client_session.py (continued)

```
            while not self.closed.is_set() and not self.server.stopping.is_set():
                try:
                    # A timeout lets the thread notice server shutdown without
                    # using a busy loop or blocking forever on a dead peer.
                    data = self.socket.recv(4096)
                except socket.timeout:
                    continue
                if not data:
                    break
                try:
                    # One recv may contain half a message or many messages;
                    # framing is deliberately delegated to the shared decoder.
                    messages = self.decoder.feed(data)
                except IRCStreamError as exc:
                    logger.warning("Malformed IRC input from %s: %s", self.address, exc)
                    self.server.send_error_line(self.client, "Malformed IRC line")
                    with self.send_lock:
                        self.send_lock.wait_for(
                            lambda: self._error_sent or self.closed.is_set(),
                            timeout=self.OUTPUT_QUEUE_TIMEOUT,
                        )
                    break
                for incoming in messages:
                    if self.closed.is_set():
                        break
                    self.server.handlers.handle(self.client, incoming)
        except (ConnectionError, OSError) as exc:
            logger.info("IRC client disconnected: %s (%s)", self.address, exc)
        except Exception:
            # An unexpected bug must be visible during VM testing, while the
            # finally block still guarantees state and socket cleanup.
            logger.exception("Unhandled IRC session error from %s", self.address)
        finally:
            # All exits, including malformed input and TCP resets, pass through
            # the same cleanup path so channel membership cannot become stale.
            self.server.disconnect(self.client, "Connection closed", announce=True)
            if self._writer is not None and self._writer.ident is not None:
                self._writer.join(timeout=2)

    def close(self) -> None:
        with self.send_lock:
            if self.closed.is_set():
                return
            self.closed.set()
            # Discard pending output and wake an idle writer immediately.
            while True:
                try:
                    self._outgoing.get_nowait()
                except Empty:
                    break
            self._outgoing.put_nowait(None)
            self.send_lock.notify_all()
        try:
            self.socket.shutdown(socket.SHUT_RDWR)
        except OSError as exc:
            logger.debug(
                "Socket shutdown already complete for %s: %s", self.address, exc
            )
        try:
            self.socket.close()
```

server/client_session.py (continued)

```
181         except OSError as exc:
182             logger.debug("Socket close failed for %s: %s", self.address, exc)
```

server/errors.py

```
  1  """Domain exceptions raised by the server state and command layers."""
  2
  3
  4  class NicknameInUseError(ValueError):
  5      """Raised when a client tries to claim an occupied IRC nickname."""
```

server/handlers.py

```
  1  """IRC command dispatch, validation and response routing."""
  2
  3  from __future__ import annotations
  4
  5  from datetime import datetime, timezone
  6
  7  from common import numerics
  8  from common.irc_message import IRCMessage, message
  9  from common.protocol import (
 10      CHANNEL_PREFIXES,
 11      IRC_COMMANDS,
 12      is_channel,
 13      valid_nickname,
 14      valid_username,
 15  )
 16
 17  from .errors import NicknameInUseError
 18  from .interfaces import HandlerServer
 19  from .state import ClientState
 20
 21
 22  class IRCHandlers:
 23      def __init__(self, server: HandlerServer) -> None:
 24          self.server = server
 25
 26      def handle(self, client: ClientState, incoming: IRCMessage) -> None:
 27          if client.disconnected:
 28              raise ConnectionError("client has disconnected")
 29          client.last_activity = self.server.monotonic()
 30          command = incoming.command
 31          if command not in IRC_COMMANDS:
 32              numeric = (
 33                  numerics.ERR_NOTREGISTERED
 34                  if not client.registered
 35                  else numerics.ERR_UNKNOWNCOMMAND
 36              )
 37              self.error(
 38                  client,
 39                  numeric,
 40                  command,
 41                  "Unknown command" if client.registered else "You have not registered",
 42              )
 43              return
 44          if (
 45              command not in {"NICK", "USER", "QUIT", "PING", "PONG"}
```

server/handlers.py (continued)

```
46              and not client.registered
47          ):
48              self.error(client, numerics.ERR_NOTREGISTERED, command)
49              return
50          # Keep each command in its own method so protocol parsing stays out of
51          # socket code and individual IRC behaviors can be tested independently.
52          handler = getattr(self, f"cmd_{command.lower()}", None)
53          if handler is None:
54              self.error(client, numerics.ERR_UNKNOWNCOMMAND, command, "Unknown command")
55              return
56          # Expected input errors are handled by each command and become IRC
57          # numerics. Unexpected programming errors are allowed to reach the
58          # session boundary, where they are logged with their traceback.
59          handler(client, incoming.params)
60
61      def error(self, client: ClientState, numeric: str, *params: str) -> None:
62          target = client.nick or "*"
63          # Invalid input must not become an invalid middle field in the error reply.
64          params = tuple(
65              "*"
66              if index < len(params) - 1
67              and (
68                  not value
69                  or len(value.encode("utf-8")) > 100
70                  or value.startswith(":")
71                  or any(char.isspace() for char in value)
72              )
73              else value
74              for index, value in enumerate(params)
75          )
76          self.server.send(
77              client, message(numeric, target, *params, prefix=self.server.server_name)
78          )
79
80      def cmd_nick(self, client: ClientState, params: tuple[str, ...]) -> None:
81          if not params or not params[0]:
82              self.error(client, numerics.ERR_NONICKNAMEGIVEN, "No nickname given")
83              return
84          nick = params[0]
85          if not valid_nickname(nick):
86              self.error(
87                  client, numerics.ERR_ERRONEUSNICKNAME, nick, "Erroneous nickname"
88              )
89              return
90          try:
91              old = self.server.set_nickname(client, nick)
92          except NicknameInUseError:
93              self.error(
94                  client, numerics.ERR_NICKNAMEINUSE, nick, "Nickname is already in use"
95              )
96              return
97          if client.registered and old is None:
98              self.welcome(client)
99
100     def cmd_user(self, client: ClientState, params: tuple[str, ...]) -> None:
101         if client.username is not None:
102             self.error(client, numerics.ERR_ALREADYREGISTERED, "You may not reregister")
103             return
104         if len(params) < 4:
105             self.error(
```

server/handlers.py (continued)

```
106                 client, numerics.ERR_NEEDMOREPARAMS, "USER", "Not enough parameters"
107             )
108             return
109         if not valid_username(params[0]):
110             self.error(
111                 client,
112                 numerics.ERR_NEEDMOREPARAMS,
113                 "USER",
114                 "Invalid username: use 1-30 printable ASCII characters excluding ! @ :",
115             )
116             return
117         self.server.state.set_user(client, params[0], params[3])
118         if client.registered:
119             self.welcome(client)
120 
121     def welcome(self, client: ClientState) -> None:
122         nick = client.nick or "*"
123         prefix = self.server.server_name
124         replies = (
125             message(
126                 numerics.RPL_WELCOME,
127                 nick,
128                 f"Welcome to {self.server.server_name}",
129                 prefix=prefix,
130             ),
131             message(
132                 numerics.RPL_YOURHOST,
133                 nick,
134                 f"Your host is {self.server.server_name}",
135                 prefix=prefix,
136             ),
137             message(
138                 numerics.RPL_CREATED,
139                 nick,
140                 "IRC server ready",
141                 prefix=prefix,
142             ),
143             message(
144                 numerics.RPL_MYINFO,
145                 nick,
146                 self.server.server_name,
147                 "irc-mini",
148                 "io",
149                 "",
150                 prefix=prefix,
151             ),
152         )
153         for reply in replies:
154             self.server.send(client, reply)
155 
156     def cmd_join(self, client: ClientState, params: tuple[str, ...]) -> None:
157         if not params or not params[0]:
158             self.error(
159                 client, numerics.ERR_NEEDMOREPARAMS, "JOIN", "Not enough parameters"
160             )
161             return
162         for channel in params[0].split(","):
163             if not is_channel(channel):
164                 self.error(
165                     client, numerics.ERR_NOSUCHCHANNEL, channel, "No such channel"
```

server/handlers.py (continued)

```
166                 )
167                 continue
168             if not self.server.join_channel(client, channel):
169                 continue
170             # JOIN is echoed to the joining client as well as existing members;
171             # NAMES then supplies the initial roster for clients joining late.
172             if client.disconnected:
173                 return
174             self.send_names(client, channel)
175
176     def cmd_part(self, client: ClientState, params: tuple[str, ...]) -> None:
177         if not params or not params[0]:
178             self.error(
179                 client, numerics.ERR_NEEDMOREPARAMS, "PART", "Not enough parameters"
180             )
181             return
182         reason = params[1] if len(params) > 1 else "Leaving"
183         for channel in params[0].split(","):
184             if not is_channel(channel):
185                 self.error(
186                     client, numerics.ERR_NOSUCHCHANNEL, channel, "No such channel"
187                 )
188                 continue
189             if not self.server.part_channel(client, channel, reason):
190                 self.error(
191                     client,
192                     numerics.ERR_NOTONCHANNEL,
193                     channel,
194                     "You are not on that channel",
195                 )
196                 continue
197
198     def cmd_names(self, client: ClientState, params: tuple[str, ...]) -> None:
199         channels = (
200             params[0].split(",")
201             if params and params[0]
202             else self.server.state.channels_for(client)
203         )
204         for channel in channels:
205             if not is_channel(channel):
206                 self.error(
207                     client, numerics.ERR_NOSUCHCHANNEL, channel, "No such channel"
208                 )
209                 continue
210             self.send_names(client, channel)
211
212     def send_names(self, client: ClientState, channel: str) -> None:
213         """Send the roster in 353 replies, then terminate it with a 366 reply.
214
215         Split at nickname boundaries using the encoded reply size, including
216         the server prefix, routing fields, and CRLF, against the 512-byte limit.
217         """
218         names = sorted(
219             member.nick or "*" for member in self.server.state.channel_members(channel)
220         )
221         nick = client.nick or "*"
222         prefix = self.server.server_name
223         chunks: list[str] = []
224         current: list[str] = []
225         for name in names:
```

server/handlers.py (continued)

```
            candidate = " ".join((*current, name))
            reply = message(
                numerics.RPL_NAMREPLY, nick, "=", channel, candidate, prefix=prefix
            )
            if current and len(reply.to_bytes()) > 512:
                chunks.append(" ".join(current))
                current = [name]
            else:
                current.append(name)
        chunks.append(" ".join(current))
        for chunk in chunks:
            if client.disconnected:
                return
            self.server.send_reply(
                client,
                message(
                    numerics.RPL_NAMREPLY, nick, "=", channel, chunk, prefix=prefix
                ),
            )
        self.server.send_reply(
            client,
            message(
                numerics.RPL_ENDOFNAMES,
                nick,
                channel,
                "End of /NAMES list.",
                prefix=prefix,
            ),
        )

    def cmd_privmsg(self, client: ClientState, params: tuple[str, ...]) -> None:
        if len(params) < 2:
            self.error(
                client, numerics.ERR_NEEDMOREPARAMS, "PRIVMSG", "Not enough parameters"
            )
            return
        target, text = params[0], params[1]
        if not text:
            self.error(client, "412", "No text to send")
            return
        outgoing = message("PRIVMSG", target, text, prefix=client.prefix)
        if len(outgoing.to_bytes()) > 512:
            self.error(client, numerics.ERR_INPUTTOOLONG, "PRIVMSG", "Message too long")
            return
        if target.startswith(CHANNEL_PREFIXES):
            if not is_channel(target):
                self.error(
                    client, numerics.ERR_NOSUCHCHANNEL, target, "No such channel"
                )
                return
            members = self.server.state.channel_members(target)
            if client not in members:
                self.error(
                    client,
                    numerics.ERR_CANNOTSENDTOCHAN,
                    target,
                    "Cannot send to channel",
                )
                return
            # IRC channel messages are delivered to peers, not echoed back to
```

server/handlers.py (continued)

```
286             # the sender; the client already knows what it submitted.
287             self.server.send_many(
288                 (member for member in members if member is not client), outgoing
289             )
290             return
291         destination = self.server.state.direct_target(target)
292         if destination is None:
293             self.error(client, numerics.ERR_NOSUCHNICK, target, "No such nick")
294             return
295         self.server.send(destination, outgoing)
296 
297     def cmd_ping(self, client: ClientState, params: tuple[str, ...]) -> None:
298         token = params[0] if params else self.server.server_name
299         reply = message(
300             "PONG", self.server.server_name, token, prefix=self.server.server_name
301         )
302         if len(reply.to_bytes()) > 512:
303             # Optional routing fields must not displace the echoed PING token.
304             reply = message("PONG", token)
305         self.server.send(client, reply)
306 
307     def cmd_pong(self, client: ClientState, params: tuple[str, ...]) -> None:
308         if params and params[-1] == self.server.server_name:
309             self.server.state.acknowledge_ping(client)
310 
311     def cmd_time(self, client: ClientState, params: tuple[str, ...]) -> None:
312         now = datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M:%S UTC")
313         self.server.send(
314             client,
315             message(
316                 numerics.RPL_TIME,
317                 client.nick or "*",
318                 self.server.server_name,
319                 now,
320                 prefix=self.server.server_name,
321             ),
322         )
323 
324     def cmd_ison(self, client: ClientState, params: tuple[str, ...]) -> None:
325         if not params:
326             self.error(
327                 client, numerics.ERR_NEEDMOREPARAMS, "ISON", "Not enough parameters"
328             )
329             return
330         names: list[str] = []
331         seen: set[str] = set()
332         nick = client.nick or "*"
333         prefix = self.server.server_name
334         for name in (name for param in params for name in param.split()):
335             target = self.server.state.direct_target(name)
336             if target is None or not target.nick or target.nick in seen:
337                 continue
338             names.append(target.nick)
339             seen.add(target.nick)
340         reply = message("303", nick, " ".join(names), prefix=prefix)
341         if len(reply.to_bytes()) > 512:
342             self.error(
343                 client,
344                 numerics.ERR_INPUTTOOLONG,
345                 "ISON",
```

server/handlers.py (continued)

```
                "Reply too long; use smaller batches",
            )
            return
        self.server.send(client, reply)

    def cmd_list(self, client: ClientState, params: tuple[str, ...]) -> None:
        requested = params[0].split(",") if params else None
        for name, count in self.server.state.list_channels(requested):
            if client.disconnected:
                return
            self.server.send_reply(
                client,
                message(
                    numerics.RPL_LIST, client.nick or "*", name, str(count), "",
                    prefix=self.server.server_name,
                ),
            )
        self.server.send_reply(
            client,
            message(
                numerics.RPL_LISTEND, client.nick or "*", "End of /LIST",
                prefix=self.server.server_name,
            ),
        )

    def cmd_lusers(self, client: ClientState, params: tuple[str, ...]) -> None:
        count = self.server.state.registered_user_count()
        self.server.send(
            client,
            message(
                numerics.RPL_LUSERCLIENT,
                client.nick or "*",
                f"There are {count} users and 0 services on 1 server",
                prefix=self.server.server_name,
            ),
        )

    def cmd_quit(self, client: ClientState, params: tuple[str, ...]) -> None:
        reason = params[0] if params else "Client Quit"
        self.server.disconnect(client, reason)
```

server/interfaces.py

```
"""Structural contracts for command handling and client delivery."""

from __future__ import annotations

from collections.abc import Iterable
from typing import TYPE_CHECKING, Protocol

from common.irc_message import IRCMessage

if TYPE_CHECKING:
    from .state import ClientState, IRCState


class ClientConnection(Protocol):
    """Separate immediate enqueueing from bounded reply production."""

```

server/interfaces.py (continued)

```
    def send(self, msg: IRCMessage) -> None: ...

    def send_reply(self, msg: IRCMessage) -> None: ...

    def close(self) -> None: ...


class HandlerServer(Protocol):
    state: IRCState
    server_name: str

    def monotonic(self) -> float: ...

    def send(self, client: ClientState, msg: IRCMessage) -> None: ...

    def send_reply(self, client: ClientState, msg: IRCMessage) -> None: ...

    def send_many(self, clients: Iterable[ClientState], msg: IRCMessage) -> None: ...

    def join_channel(self, client: ClientState, channel: str) -> bool: ...

    def part_channel(self, client: ClientState, channel: str, reason: str) -> bool: ...

    def set_nickname(self, client: ClientState, nick: str) -> str | None: ...

    def disconnect(
        self, client: ClientState, reason: str = "Client Quit", announce: bool = True
    ) -> None: ...
```

server/server.py

```
"""Small, concurrent IPv6 IRC server for a practical protocol subset."""

from __future__ import annotations

import logging
from collections import deque
import socket
import threading
import time
from collections.abc import Iterable

from common.irc_message import IRCMessage, message

from .client_session import ClientSession
from .handlers import IRCHandlers
from .state import ClientState, IRCState

logger = logging.getLogger(__name__)


class IRCServer:
    REGISTRATION_TIMEOUT = 30.0
    IDLE_TIMEOUT = 30.0
    PONG_TIMEOUT = 30.0

    def __init__(
        self, host: str = "::", port: int = 6667, server_name: str = "irc.local"
    ) -> None:
```

server/server.py (continued)

```
29          if not 0 <= port <= 65535:
30              raise ValueError("port must be between 0 and 65535")
31          if (
32              not 1 <= len(server_name) <= 63
33              or not server_name.isascii()
34              or any(not 33 <= ord(char) <= 126 or char == ":" for char in server_name)
35          ):
36              raise ValueError(
37                  "server_name must be 1-63 printable ASCII characters without spaces or ':'"
38              )
39          self.host = host
40          self.port = port
41          self.server_name = server_name
42          self.state = IRCState()
43          self.handlers = IRCHandlers(self)
44          self.socket: socket.socket | None = None
45          self.ready = threading.Event()
46          self.stopping = threading.Event()
47          self._monitor: threading.Thread | None = None
48
49      @staticmethod
50      def monotonic() -> float:
51          return time.monotonic()
52
53      def serve_forever(self) -> None:
54          # AF_INET6 is explicit so this server accepts IPv6 clients
55          # and bind to the unspecified address by default.
56          try:
57              # Publish the listener and readiness atomically with shutdown.
58              with self.state.locked():
59                  if self.stopping.is_set():
60                      return
61                  self.socket = socket.socket(socket.AF_INET6, socket.SOCK_STREAM)
62                  self.socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
63                  self.socket.bind((self.host, self.port))
64                  self.socket.listen(100)
65                  self.socket.settimeout(1.0)
66                  self.ready.set()
67                  self._monitor = threading.Thread(
68                      target=self._monitor_clients, name="irc-monitor", daemon=True
69                  )
70                  self._monitor.start()
71              while not self.stopping.is_set():
72                  try:
73                      sock, address = self.socket.accept()
74                  except socket.timeout:
75                      continue
76                  except OSError:
77                      if self.stopping.is_set():
78                          break
79                      raise
80                  ClientSession(self, sock, address).start()
81          finally:
82              self.stop()
83
84      def send(self, client: ClientState, msg: IRCMessage) -> None:
85          self.send_many((client,), msg)
86
87      def send_reply(self, client: ClientState, msg: IRCMessage) -> None:
88          """Allow bounded queue waits only outside a state transaction."""
```

server/server.py (continued)

```
        self.state.require_unlocked()
        if client.disconnected:
            return
        try:
            client.connection.send_reply(msg)
        except (ConnectionError, OSError) as exc:
            logger.info("Unable to send to %s: %s", client.address, exc)
            self.disconnect(client, "Connection lost")

    def send_many(self, clients: Iterable[ClientState], msg: IRCMessage) -> None:
        # Connection.send only enqueues; socket writes belong to each session.
        pending = deque([(list(clients), msg)])
        with self.state.locked():
            while pending:
                recipients, outgoing = pending.popleft()
                for client in recipients:
                    if client.disconnected:
                        continue
                    try:
                        client.connection.send(outgoing)
                    except (ConnectionError, OSError) as exc:
                        logger.info("Unable to send to %s: %s", client.address, exc)
                        peers = self._close_client(client)
                        if client.nick and peers:
                            pending.append(
                                (
                                    list(peers),
                                    message("QUIT", "Connection lost", prefix=client.prefix),
                                )
                            )

    def join_channel(self, client: ClientState, channel: str) -> bool:
        with self.state.locked():
            joined, members = self.state.join(client, channel)
            if joined:
                self.send_many(members, message("JOIN", channel, prefix=client.prefix))
        return joined

    def set_nickname(self, client: ClientState, nick: str) -> str | None:
        with self.state.locked():
            old_prefix = client.prefix
            old = self.state.set_nick(client, nick)
            if old and old != nick:
                recipients = {client}
                for channel in self.state.channels_for(client):
                    recipients.update(self.state.channel_members(channel))
                self.send_many(recipients, message("NICK", nick, prefix=old_prefix))
            return old

    def part_channel(self, client: ClientState, channel: str, reason: str) -> bool:
        with self.state.locked():
            parted, members = self.state.part(client, channel)
            if parted:
                self.send_many(
                    members, message("PART", channel, reason, prefix=client.prefix)
                )
            return parted

    def send_error_line(self, client: ClientState, detail: str) -> None:
        self.send(client, message("ERROR", detail, prefix=self.server_name))
```

server/server.py (continued)

```
149
150     def disconnect(
151         self, client: ClientState, reason: str = "Client Quit", announce: bool = True
152     ) -> None:
153         with self.state.locked():
154             recipients = self._close_client(client)
155             if announce and client.nick and recipients:
156                 self.send_many(recipients, message("QUIT", reason, prefix=client.prefix))
157
158     def _close_client(self, client: ClientState) -> set[ClientState]:
159         # Remove routes and close before any potentially blocking peer writes.
160         affected = self.state.remove(client)
161         client.connection.close()
162         return {
163             recipient
164             for _channel, channel_recipients in affected
165             for recipient in channel_recipients
166         }
167
168     def _monitor_clients(self) -> None:
169         while not self.stopping.wait(1):
170             self._check_idle_clients()
171
172     def _check_idle_clients(self) -> None:
173         now = self.monotonic()
174         with self.state.locked():
175             clients = list(self.state.clients)
176         for client in clients:
177             with self.state.locked():
178                 if client.disconnected:
179                     continue
180                 if (
181                     not client.registered
182                     and now - client.connected_at >= self.REGISTRATION_TIMEOUT
183                 ):
184                     # Keep expiration and removal atomic with NICK/USER updates.
185                     self.disconnect(client, "Registration timeout", announce=False)
186                     continue
187                 send_probe = (
188                     client.ping_sent is None
189                     and not client.ping_queued
190                     and now - client.last_activity >= self.IDLE_TIMEOUT
191                 )
192                 if send_probe:
193                     client.ping_queued = True
194                 timed_out = (
195                     client.ping_sent is not None
196                     and now - client.ping_sent >= self.PONG_TIMEOUT
197                 )
198             if send_probe:
199                 self.send(
200                     client, message("PING", self.server_name, prefix=self.server_name)
201                 )
202             if timed_out:
203                 self.disconnect(client, "Ping timeout")
204
205     def stop(self) -> None:
206         with self.state.locked():
207             if self.stopping.is_set():
208                 return
```

server/server.py (continued)

```
209             self.stopping.set()
210             self.ready.clear()
211             clients = list(self.state.clients)
212         if self.socket:
213             try:
214                 self.socket.close()
215             except OSError as exc:
216                 logger.debug("Server listener close failed: %s", exc)
217         for client in clients:
218             self.disconnect(client, "Server shutting down", announce=False)
```

server/state.py

```
  1 """Thread-safe in-memory IRC users, channels and routing state."""
  2
  3 from __future__ import annotations
  4
  5 import threading
  6 import time
  7 from collections.abc import Iterator
  8 from contextlib import contextmanager
  9 from dataclasses import dataclass, field
 10
 11 from .errors import NicknameInUseError
 12 from .interfaces import ClientConnection
 13 from common.protocol import irc_casefold
 14
 15
 16 @dataclass(eq=False)
 17 class ClientState:
 18     connection: ClientConnection
 19     address: tuple
 20     nick: str | None = None
 21     username: str | None = None
 22     realname: str | None = None
 23     channels: set[str] = field(default_factory=set)
 24     last_activity: float = field(default_factory=time.monotonic)
 25     ping_sent: float | None = None
 26     disconnected: bool = False
 27     connected_at: float = field(default_factory=time.monotonic)
 28     ping_queued: bool = False
 29
 30     @property
 31     def registered(self) -> bool:
 32         return bool(self.nick and self.username)
 33
 34     @property
 35     def prefix(self) -> str:
 36         nick = self.nick or "*"
 37         user = self.username or "unknown"
 38         host = self.address[0] if self.address else "unknown"
 39         return f"{nick}!{user}@{host}"
 40
 41
 42 class IRCState:
 43     """Owns all mutable server state; callers must not retain internal sets."""
 44
 45     def __init__(self) -> None:
 46         self._lock = threading.RLock()
```

server/state.py (continued)

```
47          self._lock_owner: int | None = None
48          self._lock_depth = 0
49          self.clients: set[ClientState] = set()
50          self.nicknames: dict[str, ClientState] = {}
51          self.channels: dict[str, set[ClientState]] = {}
52
53      @contextmanager
54      def locked(self) -> Iterator[None]:
55          """Keep state changes atomic and track nested ownership for delivery."""
56          with self._lock:
57              self._lock_owner = threading.get_ident()
58              self._lock_depth += 1
59              try:
60                  yield
61              finally:
62                  self._lock_depth -= 1
63                  if self._lock_depth == 0:
64                      self._lock_owner = None
65
66      def require_unlocked(self) -> None:
67          if self._lock_owner == threading.get_ident():
68              raise RuntimeError("reply delivery cannot wait while holding IRC state")
69
70      def add_client(self, client: ClientState) -> None:
71          with self.locked():
72              self._require_connected(client)
73              self.clients.add(client)
74
75      @staticmethod
76      def _channel_key(channel: str) -> str:
77          """IRC channel names are case-insensitive for routing and membership."""
78          return irc_casefold(channel)
79
80      def find_nick(self, nick: str) -> ClientState | None:
81          with self.locked():
82              return self.nicknames.get(irc_casefold(nick))
83
84      def set_nick(self, client: ClientState, nick: str) -> str | None:
85          with self.locked():
86              self._require_connected(client)
87              # IRC nickname comparisons are case-insensitive, while preserving
88              # the spelling the client chose for display and wire replies.
89              old = client.nick
90              if (
91                  irc_casefold(nick) in self.nicknames
92                  and self.nicknames[irc_casefold(nick)] is not client
93              ):
94                  raise NicknameInUseError("nickname in use")
95              if old:
96                  self.nicknames.pop(irc_casefold(old), None)
97              client.nick = nick
98              self.nicknames[irc_casefold(nick)] = client
99              return old
100
101     def set_user(self, client: ClientState, username: str, realname: str) -> None:
102         with self.locked():
103             self._require_connected(client)
104             client.username = username
105             client.realname = realname
106
```

server/state.py (continued)

```
107     def join(self, client: ClientState, channel: str) -> tuple[bool, list[ClientState]]:
108         with self.locked():
109             self._require_connected(client)
110             # Return a snapshot so the caller can release the state lock before
111             # performing potentially blocking socket writes.
112             channel_key = self._channel_key(channel)
113             members = self.channels.setdefault(channel_key, set())
114             if client in members:
115                 return False, list(members)
116             members.add(client)
117             client.channels.add(channel_key)
118             return True, list(members)
119 
120     @staticmethod
121     def _require_connected(client: ClientState) -> None:
122         if client.disconnected:
123             raise ConnectionError("client has disconnected")
124 
125     def part(self, client: ClientState, channel: str) -> tuple[bool, list[ClientState]]:
126         with self.locked():
127             channel_key = self._channel_key(channel)
128             members = self.channels.get(channel_key)
129             if not members or client not in members:
130                 return False, []
131             members.remove(client)
132             client.channels.discard(channel_key)
133             recipients = list(members) + [client]
134             if not members:
135                 self.channels.pop(channel_key, None)
136             return True, recipients
137 
138     def channel_members(self, channel: str) -> list[ClientState]:
139         with self.locked():
140             return list(self.channels.get(self._channel_key(channel), set()))
141 
142     def channels_for(self, client: ClientState) -> list[str]:
143         with self.locked():
144             return list(client.channels)
145 
146     def list_channels(self, requested: list[str] | None = None) -> list[tuple[str, int]]:
147         """Return channel counts without exposing live membership sets."""
148         keys = {irc_casefold(name) for name in requested} if requested is not None else None
149         with self.locked():
150             return sorted(
151                 (name, len(members))
152                 for name, members in self.channels.items()
153                 if keys is None or name in keys
154             )
155 
156     def registered_user_count(self) -> int:
157         with self.locked():
158             return sum(client.registered for client in self.clients)
159 
160     def acknowledge_ping(self, client: ClientState) -> None:
161         with self.locked():
162             client.ping_sent = None
163 
164     def start_ping(self, client: ClientState, now: float) -> None:
165         """Start the PONG deadline when the writer takes the queued probe."""
166         with self.locked():
```

server/state.py (continued)

```
            if not client.disconnected and client.ping_queued:
                client.ping_queued = False
                client.ping_sent = now

    def direct_target(self, nick: str) -> ClientState | None:
        with self.locked():
            client = self.nicknames.get(irc_casefold(nick))
            if client is not None and client in self.clients and client.registered:
                return client
            return None

    def remove(self, client: ClientState) -> list[tuple[str, list[ClientState]]]:
        """Remove a client and return channels plus remaining recipients."""
        with self.locked():
            # Clean every index in one critical section so no later command can
            # route a message to a disconnected session.
            client.disconnected = True
            affected: list[tuple[str, list[ClientState]]] = []
            for channel in list(client.channels):
                members = self.channels.get(channel, set())
                members.discard(client)
                affected.append((channel, list(members)))
                if not members:
                    self.channels.pop(channel, None)
            client.channels.clear()
            self.clients.discard(client)
            if client.nick and self.nicknames.get(irc_casefold(client.nick)) is client:
                self.nicknames.pop(irc_casefold(client.nick), None)
            return affected
```